Manual de Usuario: Simulador del Algoritmo del Banquero

Universidad Mariano Gálvez de Guatemala — Facultad de Ingeniería · Sistemas Operativos 1

Programa: `banquero.py`

1. ¿Para qué sirve?

Este programa revisa si un sistema operativo puede repartir sus recursos entre varios procesos sin caer en un interbloqueo (deadlock). Usted escribe los datos del sistema y el programa responde si el estado es **seguro** o **inseguro** y, cuando es seguro, en qué orden se pueden atender los procesos. También permite probar qué pasa si un proceso pide más recursos.

2. Qué necesita

- Python 3.6 o superior instalado. Para comprobarlo escriba `python3 --version` en una terminal (en Windows: `python --version`).
- El archivo `banquero.py`.
- No necesita instalar nada más ni tener internet.

3. Cómo iniciar y cerrar el programa

1. Abra una terminal en la carpeta donde guardó `banquero.py`.
2. Escriba `python3 banquero.py` y presione Enter. En Windows también puede usar `python banquero.py` o `py banquero.py`.
3. Aparece el menú principal. Para elegir una opción escriba su número y presione Enter.
4. Para salir escriba `0` en el menú (o presione Ctrl+C en cualquier momento).

4. Palabras que verá en pantalla

- **Proceso (P0, P1, ...)**: programa en ejecución. Se numeran desde 0.
- **Recurso (R0, R1, ...)**: tipo de recurso del sistema (por ejemplo impresoras o discos). De cada tipo hay varias unidades.
- **N** y **M**: cantidad de procesos y cantidad de tipos de recursos.
- **Allocation**: cuántas unidades de cada recurso tiene ahora cada proceso.

- **Max**: cuántas unidades de cada recurso necesitará como máximo cada proceso.
- **Need**: lo que a cada proceso todavía le falta pedir (Max − Allocation). El programa lo calcula solo.
- **Available**: unidades libres de cada recurso en este momento.
- **Estado seguro**: existe un orden en el que todos los procesos pueden terminar. **Estado inseguro**: ese orden no existe y puede haber interbloqueo.
- **Secuencia segura**: ese orden, por ejemplo P1 -> P3 -> P0.
- **Request**: solicitud de recursos que hace un proceso.

5. Menú principal

```
1. Ingresar datos manualmente
2. Cargar un escenario de prueba
3. Ver el estado actual (Allocation, Max, Need, Available)
4. Ejecutar el algoritmo de seguridad
5. Simular una solicitud de recursos (Request)
0. Salir
```

Antes de usar las opciones 3, 4 o 5 debe haber datos cargados con la opción 1 o con la 2.

6. Cómo usar cada opción

Opción 1: Ingresar datos manualmente

1. Escriba el número de procesos (N) y presione Enter.
2. Escriba el número de tipos de recursos (M) y presione Enter.
3. Matriz **Allocation**: el programa pide una línea por proceso. En cada línea escriba M números separados por espacios (también sirven las comas) y presione Enter.
4. Matriz **Max**: igual que el paso anterior.
5. Vector **Available**: escriba en una sola línea M números.
6. El programa confirma los datos y muestra las tablas Allocation, Max, Need y Available.

Ejemplo con 3 procesos y 2 recursos (lo que usted escribe aparece después de los dos puntos):

```
Número de procesos (N): 3
Número de tipos de recursos (M): 2

Matriz Allocation: escriba 2 valores por cada proceso (P0 a P2).
  Allocation de P0: 1 0
  Allocation de P1: 0 1
  Allocation de P2: 1 1

Matriz Max: escriba 2 valores por cada proceso (P0 a P2).
  Max de P0: 2 1
  Max de P1: 1 2
  Max de P2: 2 2

Vector Available (instancias libres de cada recurso): 1 1
```

Con esos datos el programa calcula y muestra Need:

```
Need = Max - Allocation (lo que aún puede pedir):
          R0  R1
  P0       1   1
  P1       1   1
  P2       1   1
```

Opción 2: Cargar un escenario de prueba

Muestra una lista de escenarios ya preparados. Escriba `1` para un caso seguro (5 procesos y 3 recursos) o `2` para un caso inseguro (4 procesos y 3 recursos). Luego se muestran las tablas del escenario. Sirve para probar el programa sin escribir datos y se puede usar en cualquier momento, sin reiniciar.

Opción 3: Ver el estado actual

Muestra otra vez las tablas Allocation, Max, Need y Available con los datos que están cargados. Úsela para revisar lo que escribió o para ver cómo cambió el sistema después de conceder una solicitud.

Opción 4: Ejecutar el algoritmo de seguridad

El programa muestra el paso a paso y, al final, el resultado. Ejemplo con el escenario 1:

```
  Ronda 1  (Work = [3, 3, 2])
    P0: Need [7, 4, 3] NO es <= Work [3, 3, 2] -> espera
    P1: Need [1, 2, 2] <= Work [3, 3, 2] -> TERMINA y libera [2, 0, 0]
        Work = [3, 3, 2] + [2, 0, 0] = [5, 3, 2]
```

Cómo leerlo:

- **Ronda**: una vuelta por los procesos que aún no han terminado. Se indica el valor de Work, que son los recursos disponibles en ese momento.
- **TERMINA y libera [...]**: el proceso tiene recursos suficientes (su Need cabe en Work), termina y devuelve lo que tenía. La línea siguiente muestra el nuevo Work.
- **espera**: al proceso le falta algún recurso por ahora; se vuelve a evaluar en la siguiente ronda.
- Al final aparece el **resultado** dentro de un recuadro de signos `=` (ver la sección 7).

Opción 5: Simular una solicitud de recursos (Request)

1. Escriba el número del proceso que hace la solicitud (de 0 a N−1) y presione Enter.
2. Escriba el vector Request: M números separados por espacios, con la cantidad que pide de cada recurso.
3. Lea la respuesta del programa:

- **SOLICITUD RECHAZADA**: el proceso pide más de lo que declaró en Max (excede su Need). No se cambia nada.
- **DEBE ESPERAR**: no hay suficientes recursos libres en este momento. No se cambia nada.
- **SOLICITUD DENEGADA**: aunque hay recursos, concederla dejaría el sistema en estado inseguro. No se cambia nada.
- **SOLICITUD CONCEDIDA**: el estado resultante es seguro. La solicitud **sí se aplica**: Available baja, Allocation sube y Need baja. El programa muestra una secuencia segura y el nuevo estado del sistema.

Para volver a los datos originales después de conceder una solicitud, cargue el escenario de nuevo (opción 2) o vuelva a ingresar los datos (opción 1). Si todos los valores del Request son 0, el programa avisa que no hay nada que evaluar.

Opción 0: Salir

Cierra el programa.

7. Cómo interpretar el resultado

- **ESTADO SEGURO**: todos los procesos pueden terminar. Se muestra una secuencia segura, por ejemplo `P1 -> P3 -> P4 -> P0 -> P2`, que es un orden válido para atenderlos. Puede haber otras secuencias válidas; el programa muestra una.

- **ESTADO INSEGURO (posible interbloqueo)**: no existe un orden en el que todos terminen. El programa lista los procesos que no pueden terminar y, si los hay, los que sí alcanzaron a terminar.

8. Reglas para escribir los datos y mensajes de error

Todos los números deben ser enteros mayores o iguales a 0 (N y M, mayores o iguales a 1), y Allocation nunca puede ser mayor que Max. Si se equivoca, el programa muestra un mensaje y vuelve a pedir el dato:

- `Error: escriba un número entero.` Escribió letras, símbolos o decimales.
- `Error: el valor debe ser mayor o igual a 1.` N y M deben ser al menos 1.
- `Error: se esperaban X valores y se recibieron Y.` Cada línea debe tener exactamente M números.
- `Error: no se permiten valores negativos.`
- `Error: Allocation[P0][R0] = 3 supera a Max[P0][R0] = 2.` Un proceso no puede tener asignado más que su máximo; se piden de nuevo ambas matrices.
- `Primero debe ingresar datos (opción 1) o cargar un escenario (opción 2).` Use una de esas opciones antes de la 3, 4 o 5.
- `Opción no válida. Escriba un número del 0 al 5.`

Si en Windows aparece que `python` no se reconoce, pruebe con `py banquero.py` o reinstale Python marcando la casilla "Add Python to PATH".